ISTQB® Certified Tester

Advanced Level — **Security Tester** (CT-SEC, Syllabus 2016)

CHAPTER 3

Security Testing Processes

Complete Study Guide

Chapter fact	Value
Syllabus sections covered	3.1 Security Test Process Definition · 3.2 Security Test Planning · 3.3 Security Test Design · 3.4 Security Test Execution · 3.5 Security Test Evaluation · 3.6 Security Test Maintenance
Learning objectives	8 examinable (AS-3.1.1 – AS-3.6.1) · K2 = 3 · K3 = 3 · K4 = 2
Recommended teaching time	140 minutes — **12 points**, 5.1 points per teaching hour
Questions in the exam	**6 of 45 questions · 12 of 80 points.** K2 = 2 · K3 = 2 · **K4 = 2**. Guaranteed: **AS-3.1.1** (K3), **AS-3.2.1** (K4), **AS-3.4.2** (K2), **AS-3.5.1** (K4). One K3 from 3.3.1/3.3.2; one K2 from 3.4.1/3.6.1
Exam format	45 questions · **80 points** · **120 minutes** (150 with the 25% non-native-language allowance) · pass mark 65% = **52 points** · K2 = 1 pt · K3 = 2 pts · K4 = 3 pts
Keywords	account harvesting · password cracking · social engineering · test approach · test plan · test process
Official sample-exam questions	**#11 – #16** (2 + 3 + 2 + 1 + 1 + 3 points)

Built exclusively from the ISTQB® Advanced Level Security Tester syllabus (2016), the official Sample Exam (questions and answers, GA March 2016) and the ISTQB® exam structure tables.

Study this single file to master Chapter 3.

Contents

Section 1 — Syllabus Explanation....4
AS-3.1.1 (K3) — Security Test Process Definition....4
AS-3.2.1 (K4) — Security Test Planning....6
AS-3.3.1 (K3) — Security Test Design....7
AS-3.3.2 (K3) — Security Test Design Based on Policies and Procedures....10
AS-3.4.1 (K2) — An Effective Security Test Environment....11
AS-3.4.2 (K2) — Planning and Approvals Before Any Security Test....11
AS-3.5.1 (K4) — Security Test Evaluation....12
AS-3.6.1 (K2) — Security Test Maintenance....13
Section 2 — Key Concepts....14
2.1 The storyline of Chapter 3....14
2.2 The process and the lifecycle....14
2.3 Planning....15
2.4 Design....16
2.5 Execution, evaluation and maintenance....18
2.6 Fuzzy points clarified....20
2.7 Chapter 3 keyword glossary....21
Section 3 — Tables and Summaries....22
Section 4 — Demo Questions with Answers....24
Part A — The 6 official Chapter 3 questions....24
Part B — 20 additional syllabus-derived questions....26
Most Important Questions From This Chapter....31
Section 5 — Exam Tricks....33
5.1 Frequently tested topics (evidence-based)....33
5.2 Common distractors observed....33
5.3 Keyword recognition (map words → answers)....33
5.4 Typical traps....34
5.5 How to eliminate wrong answers....34
5.6 Scenario-solving shortcuts....34
5.7 Common candidate mistakes....34
5.8 Mnemonics that carry the whole chapter....35
5.9 Chapter 3 mind map....35
Section 6 — Revision Notes....36
6.1 One-page chapter summary....36
6.2 Key definitions to memorize....36
6.3 High-priority concepts (study first)....36
6.4 Last-minute revision sheet....36
6.5 Memorization checklist....37
6.6 Exam-day quick review....37

In Word, right-click the table of contents and choose "Update Field" → "Update entire table" to refresh the page numbers.

How to use this guide

Section 1 explains all eight learning objectives — read it once, slowly. **Section 2** explains every key concept from the ground up — plain words, how it works, a worked example, the exam angle, and what it is easily confused with — and ends with the chapter's **fuzzy points clarified**. If the syllabus wording in Section 1 feels dense, read the matching Section 2 card first. **Section 3** holds the tables to reproduce from memory; **table 3.1 (activity → process step)**, **table 3.2 (plan checklist)** and **table 3.5 (first response to a vulnerability)** pay directly. **Section 4** is the question bank, **Section 5** exam tactics, **Section 6** is the revision sheet for the night before.

What makes Chapter 3 unusual: it is the Foundation Level test process re-read through a security lens, but it carries **two guaranteed 3-point K4 questions** — reviewing a **security test plan** and handling a **discovered vulnerability**. Both are pattern questions with a fixed method.

Two rules that decide those six points: first, a plan is reviewed against **six elements — scope, who, schedule, tasks and time, environment, authorizations** — and the missing one is the answer. Second, when testing an update finds a vulnerability, **check production first, record it in a secure tracker, notify the developer, test for more** — and **never** put it in a general stakeholder report.

Suggested schedule: Day 1 — Section 1 §3.1 and §3.2. Day 2 — Section 1 §3.3 (the longest part: design categories, secure coding, DoS family). Day 3 — Section 1 §3.4–3.6 and Section 3. Day 4 — Section 4 and Section 5. Day 5 onwards — redo the questions you missed and re-read Section 2's fuzzy points. Night before — Section 6.

Section 1 — Syllabus Explanation

Chapter 3, **Security Testing Processes**, carries **8 learning objectives** across six sub-chapters (3.1 Process Definition · 3.2 Planning · 3.3 Design · 3.4 Execution · 3.5 Evaluation · 3.6 Maintenance). The Exam Structure Table allocates **6 questions** and **12 of the 80 points**. Teaching time is **140 minutes — 5.1 points per teaching hour**.

This is the **process** chapter: the test process from Foundation Level, re-read through a security lens. It has **two guaranteed 3-point K4 questions — analyzing a test plan (AS-3.2.1)** and **analyzing test results / incidents (AS-3.5.1)** — and one guaranteed K3 on **defining the process (AS-3.1.1)**. Those three questions are **8 of the 12 points**.

LO ID	K	Objective (syllabus statement)	Qs	Pts
AS-3.1.1	**K3**	For a given project, demonstrate the ability to define the elements of an effective security test process.	**1**	**2**
AS-3.2.1	**K4**	Analyze a given security test plan, giving feedback on strengths and weaknesses of the plan.	**1**	**3**
AS-3.3.1	**K3**	For a given project, implement conceptual (abstract) security tests, based on a given security test approach, along with identified functional and structural security risks.	1 of 2†	2
AS-3.3.2	**K3**	Implement test cases to validate security policies and procedures.	1 of 2†	2
AS-3.4.1	K2	Understand the key elements and characteristics of an effective security test environment.	1 of 2‡	1
AS-3.4.2	K2	Understand the importance of planning and obtaining approvals before performing any security test.	**1**	1
AS-3.5.1	**K4**	Analyze security test results to determine: nature, extent and potential impact of the vulnerability; suggested remediation; optimal test reporting methods.	**1**	**3**
AS-3.6.1	K2	Understand the importance of maintaining security testing processes given the evolving nature of technology and threats.	1 of 2‡	1
Chapter 3 total	—	**K2 = 2 · K3 = 2 · K4 = 2**	**6**	**12**

*† One K3 from AS-3.3.1 / 3.3.2 — the Sample Exam drew **3.3.2** (Q13). ‡ One K2 from AS-3.4.1 / 3.6.1 — the Sample Exam drew **3.4.1** (Q14).*

Exam allocation you must know

Chapter 3 = **6 questions / 12 points**; in the Sample Exam questions **#11 to #16** (2 + 3 + 2 + 1 + 1 + 3 points). With Chapter 4 it forms the **process core** of the syllabus — **28 points between them, over a third of the paper**.

The one thing that decides this chapter

Security test results are sensitive, and the first question about any vulnerability is "is it in production?" The 3-point AS-3.5.1 item (Sample Q16) is answered by: **test whether the problem exists in the current production version, document the defect in a secure system, notify the developer, continue testing for similar defects.** Any option that puts the vulnerability into a **general stakeholder report** is wrong. The same instinct answers Chapter 6's man-in-the-middle question and Chapter 7's confidentiality questions.

AS-3.1.1 (K3) — Security Test Process Definition

Original syllabus statement

For a given project, demonstrate the ability to define the elements of an effective security test process.

Security testing is a lifecycle activity. Failing to implement and test security defenses throughout a project can lead to **severe security defects which may never be fully resolved**. The security test process **must be aligned with the development process**

so activities happen when needed, and **defined in the context of** the organization's nature, technical environments, development process and business risks.

Table 3.1 — The ISTQB Security Testing Process

ISTQB test process	Security test process — goal	Example security testing tasks
Test Planning and Control	**Security Test Planning and Control** — define **an appropriate scope of testing that corresponds to the security risks**.	Define security test **objectives** · define the **scope** · identify **resources** · define **estimates and schedules** · define **metrics, entry and exit criteria** · **monitor** progress and results · **take actions** in response to information from other activities.
Test Analysis and Design	**Security Test Analysis and Design** — gain understanding of **specific threats and risks** based on **security assessments, audits and standard sources of known vulnerabilities**.	**Review the basis** (risk assessments, security requirements, security policies) · define **test conditions** based on objectives, risks, standards and known vulnerabilities, implemented defenses, scope, **applicability of security test tools**.
Test Implementation and Execution	**Security Test Implementation and Execution — translate conceptual tests into executable tests** (manual or tool), performed from **a variety of perspectives — internal user, external user, malicious user**, etc.	**Create** test cases, scenarios, scripts · perform **functional security tests** from specifications · perform **functional security and penetration testing based on tester knowledge and intuition** · test **based on a model** · **set up the test environment**.
Evaluating Exit Criteria and Reporting	**Evaluating and Reporting Results** — often **alongside execution**, to evaluate individual tests and **report new threats as soon as possible**.	Determine **specific vulnerabilities** from outcomes · evaluate **security risk levels** · report **interim and final** results to **management and other authorized parties**.
Test Closure	**Test Closure** — bring activities to closure so tests **can be maintained and performed regularly**; **all testware and results stored in a secured way** yet available for future use.	Ensure **all planned tests performed** · confirm **deliverables (reports) delivered** · **archive results, test data and sensitive information in secure locations** · **analyze results to improve development** in terms of security.

- The process is **not necessarily sequential** — it **aligns with the organization's lifecycle**, and security activities run **alongside and during** other project activities and tests.
- Table 3.1's tasks are **examples, not prescriptive requirements**; actual tasks depend on the **strategy and approach** adopted.

Figure 3.1 — Hierarchy of security test planning

Level	Element
Organizational	**Security test policy**
Organizational	**Security test strategy**
Project	**Security test approach**
Project	**Security test tasks**

How the official exam used this objective (K3, 2 points)

Sample Q11: you **use conceptual tests as a basis for manual tests and execute them from an external vendor's perspective**. Which process step? **Security test implementation.** Official rationale: turning conceptual tests into manual tests and executing them **is part of implementation**; creating conditions and objectives and analysis/design **have already been done** (the conceptual tests exist); evaluating and reporting comes **after execution**. Key phrase: **"translate conceptual tests into tests that can be executed ... from a variety of perspectives"**.

§3.1.2 — Aligning the process to a lifecycle model

Lifecycle	What the security tester should be aware of
Sequential	Security needs and risks **defined early**, documented in **software requirement specifications** · needs **may**

Lifecycle	What the security tester should be aware of
	change but not be reflected in updated requirements, so tests may look specific and complete but **not be current** due to **late-project risks** · tests **can be done any time but commonly late** · **results may be difficult to address at the end**.
Iterative / incremental (e.g. Agile)	Needs and risks **emerge throughout**, per iteration/sprint, in requirements, **user stories, models, acceptance criteria, prototypes** · **can (should) be addressed in the iteration in which they are identified** · tests **performed continuously** · some risks **may not be fully mitigated and tested in one short release cycle**.
COTS	Often **black-box**, possibly customized · **often contain vulnerabilities** requiring **frequent updates and patches** · **no access to code — structural analysis and structural testing are not possible**.
Open source	A **variant of COTS** — but **the code is available for all the world to see** · has vulnerabilities, so **patches must be kept up to date** · **once a vulnerability is publicized, users of that version (and earlier) are at risk**.

Example — security testing in a sequential lifecycle

Phase	Security testing tasks
Requirements	Security requirements **defined and reviewed**; use cases written; **a security test approach should be developed**.
Analysis and design	BA refines requirements; architect proposes a solution with security as a functional and non-functional need; testers learn the architecture; **major security test objectives defined**.
Detailed design	UIs and databases designed; security test design becomes more detailed; **first security tests may be performed based on models**.
Coding / implementation	**First opportunity to test the structure** — buffer overflow, field-level edits that might permit **SQL injection**; **static analysis and code reviews from the security perspective**; component testing; small-assembly integration.
System testing	Systems and sub-systems incl. hardware, data, procedures and people; often **transactional**; system integration testing; a **wider view** — **transaction security** incl. authentication, data storage, firewall implementation, procedural controls.
User acceptance testing	Validates real-world business processes; **security should already be largely performed**, but **business-process-level security scenarios** remain.
Deployment	Pilot, massive or **parallel** deployment; risk that **security configurations are not correct in the target environment** — e.g. **database access rights wrong in live**.
Maintenance	Test changes and **regression**; ensure **no new vulnerabilities**; keep **firewalls and security technology current**; **continuous monitoring** for suspicious activity.

Iterative example. Four-week releases from a prioritized backlog of user stories with acceptance criteria. First iteration delivers **four security functions** needed by many later features: **user login · SSL enablement · reset lost password · lockout after three unsuccessful tries**. The security tester **works with the developer** to ensure correct policies and protocols are in the code and **tests functions as they are developed**; each iteration confirms controls work and **no new vulnerabilities** were introduced.

Avoid the acceptance-phase trap

The syllabus is explicit: **avoid the situation where security testing is not performed until the acceptance phase** — at the end of the project it is **especially costly and risky** to deal with defects. In both lifecycle examples **the security testing process steps are integral tasks**.

AS-3.2.1 (K4) — Security Test Planning

Original syllabus statement

Analyze a given security test plan, giving feedback on strengths and weaknesses of the plan.

Planning objectives (§3.2.1)

Security testing should focus on **two aspects**: **(1) verifying the designed security defenses are implemented and function as designed**, and **(2) verifying that no vulnerabilities are introduced during development**. All defenses should be **based on a risk analysis** — the starting point for planning. Many unintended vulnerabilities can be avoided by **QA activities and best practices** in architecture, design and coding; testing for introduced vulnerabilities **starts with an assessment of the practices used by the development team**, which may lead to **additional security tests**.

The six key plan elements and their questions (§3.2.2)

Plan element	Questions to ask
1. Scope	What is **in and out of scope**? What is **achievable** given resources, risks and time?
2. Who performs it	Do we have people with **appropriate skills**? Are we comfortable **outsourcing**? For commercial/vendor software, **which tests are the vendor's and which the customer's**? Do testers need **training on specific tools**?
3. Schedule	Which security items must be **implemented and tested before other testing** (e.g. **access rights and logins**)? **When** will features be available? **How long** will testing take?
4. Tasks and time for each	Time to **design** tests · to **evaluate and report** results · to perform **security regression tests** · to **establish the environment**.
5. Test environment(s)	**Extent** (platform, technology, size, location)? **New** environment? Which **tools** must be installed?
6. Authorizations and approvals	**Who** must authorize and approve? **When** is authorization needed? Are **budget and funding** sufficient?

Reviewing the plan

- Review for **completeness and correctness**. Because security tests are technical, a **technical review** may be most appropriate; **walkthroughs and inspections** may also suit.
- A **standard checklist** helps; feedback must be **constructive and not aimed at the producer**.
- Reviewers come from **all areas impacted** — **not necessarily security testers**; e.g. a **business-unit manager** may know risks that belong in the plan. **IT auditors and security administrators** are especially helpful because they know **policies and procedures**.

How the official exam used this objective (K4, 3 points)

Sample Q12: a plan for a medical-records system covers **scope · roles and assignments · responsibilities (vendors vs internal) · high-level schedule · environment requirements and setup · authorizations and approvals**. What is still missing to meet the syllabus minimum? **A schedule showing the time required to design, run and evaluate the security tests** — element 4, **tasks and the time for each**.

Rejected: credentials and training (may be needed but **not a minimum requirement**, and may be covered by roles), a copy of regulatory standards (**referenced, not included**), testers' contact details for a breach (**not plan-level detail**). Method: **tick the six elements off against the stem; the unticked one is the answer**.

AS-3.3.1 (K3) — Security Test Design

Original syllabus statement

For a given project, implement conceptual (abstract) security tests, based on a given security test approach, along with identified functional and structural security risks.

Design may start **from a performed risk analysis**, **from an available threat model**, or **from an ad hoc origin categorization of security risks — any of these can form a workable basis**. Ensure there are security tests **in every applicable development phase**.

Detailed tests are based on **security risks, a security test strategy and sources such as threat models**, and can be seen as **functional and structural**. For an e-commerce site, **functional security risks** include **SQL injection, account harvesting and password cracking**; a **structural** risk is a **buffer overflow** allowing access **through a memory failure**.

Five essential attributes of detailed security tests

Attribute	Meaning
Prioritized	By identified **security risks and threat models**
Traced	To defined **security requirements**
Audience-defined	For the **intended audience** — developers, functional testers, security testers
Defect-profiled	Based on **security defect profiles**
Automatable	**Designed to be automated, if applicable**

Basic flow of security test design: (1) **security test approach** (project) → (2) **risks, threat models and requirements** (project) → (3) **design techniques** based on risks, requirements and application → (4) **test cases and scenarios**. Key principle: **a design process should be able to create and implement tests based on any identified security risk, requirement, or threat.** Stay current with **standards and threat lists** (Chapter 9).

Design categories — what to test and an example of each

Category	What the tests do	Syllabus example
Functional security controls (e.g. transaction controls)	Verify controls are **in place, working correctly and effective** in **detecting and preventing unauthorized actions**	A **bank teller** may not authorize a cash withdrawal over a limit **without the head teller's approval** in the system
Functional access controls (logins, passwords, tokens)	Username and password **policies applied correctly** · access control level **appropriate for the risk** · **resistant to password cracking software**	**Account harvesting**: with a correct user name and wrong password, the **error message must not indicate which item is incorrect**
Structural access controls (user rights, encryption levels, authentication)	Test how **rights** are set for **data access, functional access and privacy levels** — set by sysadmins, security admins or DBAs, in app config or infrastructure	Create **test accounts per access level** (minimal, manager, admin) and verify a **minimal user cannot perform manager-level activities**
Secure coding practices	**Primarily static** — are developers following established security methods? Dynamic tests check **data validation and error messaging**; **dynamic memory tools** find buffer overflows	See the practices list below
Operating system access	Once an attacker reaches the OS they control **data, network access and malware**	Test the ability to **plant rootkits and other malicious code**
Language vulnerabilities	No language is inherently secure; **how the code is implemented** is the key factor	**WhiteHat 2014**: ~30,000 sites — **negligible differences** between .NET, Java, PHP, ASP, ColdFusion, Perl; SQLi and XSS **pervasive**. CERT publications, tools and **Vulnerability Notes Database**
Platform vulnerabilities (Windows, Linux, Mac OS, iOS, Android)	Ensure **platform security updates are applied promptly and across all devices**	—
External threats	DoS/DDoS and intrusions	See below
Internal threats	**The greatest threats may be internal**	Corporate espionage · info kept by outsourced staff · theft of drives · disgruntled employees (leaks, fake invoices)

Secure coding practices (the syllabus's short list)

Key principle: **many security attacks are accomplished through exploiting software defects to cause a system to behave in an unexpected way.**

Practice	Practice
Proven session management creates **random session identifiers**	**Authorization decisions only by trusted system objects** — e.g. **server side**
No secure information in error messages (system details, session IDs, account info)	**Handle errors within the application**, not via server configuration
HTTP GET requests must not include sensitive information	**Error handlers must not display stack traces** or debugging information
Log all data input validation failures	**Protect temporary sensitive data** on the server (encrypt) and **clear it when no longer needed**
No direct OS commands from the application — use **built-in APIs**	**No clear-text passwords or connection strings on clients** (cookies); prohibit embedding in Flash, compiled code, **MS viewstate**
Encrypt all sensitive data in transmission — **TLS** for HTTP; encryption for non-HTTP	**User data never passed directly to a dynamic "include"**
Sanitize and validate all user-provided data	**Strongly typed variables** where supported (numeric fields reject alpha), also in the database
Use tested, trusted, approved code under configuration management rather than new unmanaged code	**Run services with least privilege — never as root — each with its own OS account**

References: OWASP Secure Coding Practices Quick Reference Guide, CERT Top 10 Secure Coding Practices, SANS Top 25 Most Dangerous Software Errors.

External threats — DoS, DDoS and intrusion

Attack	Mechanism	Syllabus example
DoS	**Overloading** system or application resources so it is **inaccessible to legitimate users** — targets bandwidth, connectivity or specific services	—
DDoS	Launched **indirectly using other computers** — **amplification** or **botnets** (many previously compromised machines under an attacker's control, gained via **viruses or Trojans**)	—
Amplification / reflection	Abuse a protocol (**DNS, NTP**); send traffic to broadcast addresses with the **victim's spoofed source address**; responses **multiply** toward the victim	Attacker A asks system B for **all DNS records** while impersonating victim C; B **floods C** with the amplified answer
Resource exhaustion	Abuse a desired function to **consume CPU, memory, disk**	Repeated **SSL key renegotiation** requests until the server **only generates keys**
Logical DoS	Abuse **intended functionality** to lock others out	**Predictable user names + permanent lockout after three failures** → attacker locks many accounts (and indirectly DoSes the **helpdesk**)

The four levels of testing for DDoS

Level	Test
1	Ensure computers are **not infected with known malware**
2	Test the **IDS's ability to quickly identify multiple requests from a single computer** in a short time
3	Identify **configurations allowing abusable functionality** (SSL, web server, DNS)

Level	Test
4	Identify **logic defects** that can allow a DoS

Intrusion methods (someone "breaking into" a system): **social engineering · injection attacks (SQL, malicious code) · account compromise (harvesting, password reset) · exploiting known vulnerabilities (firewall, OS, framework, application) · malware · insecure configuration · authorization defects · application logic attacks** (e.g. steps **out of order** in a shopping cart to get a discount). **Intercepting a transmission sent from inside the organization to another organization is not an intrusion, but an internal breach.**

Security test format and structure

- Use the **same format as other testing**; only the **target** and the **environment** differ. Standards (**IEEE 829-2008, ISO/IEC/IEEE 29119-3**) should be **tailored**; test management tools often provide structure.
- **Test cases** are the **most standalone** form; **no sequential execution**; typically **single conditions** — e.g. **password formatting rules**.
- **Test procedures** specify **sequence**; a **test script** is an **automated test procedure**. Use procedures **when sequence is important** — e.g. **"recover lost password"**.
- **Experience-based tests** (exploratory): conditions and expected results **not defined in advance**, but **conditions tested and actual results should be recorded**.

AS-3.3.2 (K3) — Security Test Design Based on Policies and Procedures

Original syllabus statement

Implement test cases to validate security policies and procedures.

Here **policies and procedures become the test basis**, so **security tests are almost a means of security auditing**. They **should not be the only basis** — other perspectives are needed. The three goals:

1. **Understand the purpose and scope** of the policy or procedure
2. **Assess the testability** of the policy/procedure
3. **Create tests related directly** to the policy/procedure

The syllabus's worked lockout procedure — six steps

Step	Action	Expected result
1	Attempt logging in **three times unsuccessfully**	**Lock-out message on the third** attempt; further attempts receive the lock-out message
2	**Contact IT support and verify identity**	A **temporary password** is issued **to a known email address**
3	Log in with the **temporary password**	Access granted
4	**Create a new password** conforming to the password policy	Accepted — **also an opportunity to test the password policy**
5	**Log out**	—
6	Log in with the **newly created password**	Access granted

Not all policies are this testable. "Audit records contain **all audited events with date/time stamp** and are **traceable to specific individuals**…" — testable in principle, but you must **trigger a sample set of events** and **verify the logged information** (user ID, date and time) is correct.

How the official exam used this objective (K3, 2 points)

Sample Q13 offers four lockout test cases. Correct: **three unsuccessful attempts → lock-out message → temporary password**

from manager or administrator → change it at login → log out → log back in with the new password. Options (b) and (c) are wrong **because of the word "several"** — the procedure specifies **three**. (d) uses passwords **kept in a notepad on the PC** — definitely not good practice. **Exact numbers in a procedure are test data; vague quantifiers are a defect in the test case.**

AS-3.4.1 (K2) — An Effective Security Test Environment

Original syllabus statement

Understand the key elements and characteristics of an effective security test environment.

Security testing has **unique risks that require a segregated approach**, especially for **untrusted applications** (third-party, open source). Functional controls and session management may be tested in a typical integrated environment **without high risk**, but for **unknown and untrusted code** the risk of **malware corrupting a server or network** makes an **isolated or virtual** environment advisable.

The three attributes

Attribute	Meaning
1. Isolated	From other systems — **depending on the level of malware risk**
2. Complete	Reflects the **target (production)** environment in: systems under test · **OS (exact version and configuration)** · networking · middleware · desktops · mobile devices · databases · **access rights · browsers and plug-ins · co-existing applications · data (engineered, or obfuscated production data)**
3. Restorable	To **repeat tests** and **recover from corruption**

How the official exam used this objective

Sample Q14: main characteristic? **Mimics the production environment in terms of access rights** — the closer to production, the more valid the testing, **particularly for access rights and delegation settings**. Rejected: **closely tied to production** (it **shouldn't be connected**), **old OS versions** (not a main characteristic), **extra plug-ins not in production** (cause **false positives and false negatives**).

AS-3.4.2 (K2) — Planning and Approvals Before Any Security Test

Original syllabus statement

Understand the importance of planning and obtaining approvals before performing any security test.

Why approval is needed

Reason	Syllabus wording
Legality	In almost all countries it is **against the law to (try to) get access** to data systems; **in some countries even to have access to security testing tools**. The only way is a **waiver from the system or data owner and approval from your management**.
Detection	Tests may **trigger IDS alerts**; the tester may **appear to be a malicious insider** — **penetration testing especially** needs authorization.
Damage	Tests may lead to **major system failures and blackouts**; risks must be known and **precautions in place**.
Personal consequence	Without prior **and specific** authorization the tester may **violate policies and procedures** and be subject to **termination or prosecution**.

Contents of an authorization form

Field	Detail
Name of **authorizing entity**	
Names of **testing personnel and/or entity**	
Statement of work	
Dates of authorization (to/from)	
Other details	**Originating IP addresses, user accounts**, etc.
Attestations	Customer **owns** the system · has **authority** to authorize · has **backed up** all systems and data · has **tested restore** from backup · **understands the risks**
"Hold harmless" clause	For the **testing entity**
Signatures	Of a customer representative **authorized to enter into such agreements**

How the official exam used this objective

Sample Q15: significant concern when seeking approval for tools? **Some countries prohibit the use of certain security testing tools.** Rejected: a **bypass** of approval in a crisis (a fast-track makes sense; **a complete bypass is risky**), and claims that tool risks are **rarely known** or **always known** (do the due diligence in tool selection).

AS-3.5.1 (K4) — Security Test Evaluation

Original syllabus statement

Analyze security test results to determine: nature of security vulnerability · extent of security vulnerability · potential impact of security vulnerability · suggested remediation · optimal test reporting methods.

Evaluation is performed **during execution** as individual tests run — it is **the evaluation of the outcome of a security test**. When a vulnerability is found, **an incident report should be filed** containing **at a minimum**:

Minimum incident report content	Purpose
Name of tester observing the vulnerability	Accountability
Test environment where observed	Reproduction
Test steps performed	**To facilitate re-creation**
Nature of the vulnerability	What it is
Extent of the vulnerability	How far it reaches — where else it exists
Potential impact	Severity, risk
Suggested remediation course of action	Fix

Securing the reports

Use the **same incident management system** as other testing, but **assign a special category** and possibly **secure reports from unauthorized personnel**, e.g. when:

- testing is by an **independent organization** reporting into a tool with **few viewing restrictions**
- vulnerabilities are identified but **not immediately resolved**
- **internal personnel may be a potential threat** to exploit the vulnerabilities

The IT auditor should be able to decide whether to restrict access to results. A **final security test report** at the end of a major effort (e.g. system testing) **may also need to be confidential**, depending on **vulnerability resolution status**.

How the official exam used this objective (K4, 3 points)

Sample Q16: final testing of an **update to a version already in production** finds **XSS** on the e-commerce site. Proper steps? **Test if the problem exists in the current production version, document the defect in a secure system, notify the developer, continue testing for other XSS defects.**

Rationale: **first priority — is it in production?** Document **only in a secure defect tracking system**, because production may be exposed. **One XSS suggests others**, so continue. Rejected: adding it to the **stakeholder report** (a) and **weekly status report** (d) — **the defect should not be publicized**; (c) investigates extent but **omits notification, which is critical**.

AS-3.6.1 (K2) — Security Test Maintenance

Original syllabus statement

Understand the importance of maintaining security testing processes given the evolving nature of technology and threats.

- Often maintenance only means **adding new types of tests in response to new types of threats**.
- **Security testing targets and threats change daily**, so **the process must be designed to change easily**.
- **New tools appear**; testers should **keep up** and **evaluate which tools add power and flexibility**.

Section 2 — Key Concepts

Section 1 gives the syllabus wording objective by objective. This section explains what that wording means. Each card builds one idea in order: plain words first, then how it works, an example, how the exam uses it, and what it is easily confused with.

2.1 The storyline of Chapter 3

Chapter 3 takes the **Foundation Level test process** you already know (planning and control → analysis and design → implementation and execution → evaluating exit criteria and reporting → closure) and asks, at each step, **"what changes when the thing being tested is security?"** Three answers run through the whole chapter:

4. **Security testing is dangerous.** It can break systems, trigger alarms and break the law. So it needs an **isolated environment** and **written authorization before you start**.
5. **Security results are dangerous.** A vulnerability report is a map for an attacker. So results go into a **secure tracker** for **need-to-know** readers, and the **first** question about any finding is **"does this exist in production?"**
6. **Security never stands still.** Threats change daily, so the process must be **easy to change**, and tests must be maintained.

2.2 The process and the lifecycle

The five steps of the security test process (AS-3.1.1, guaranteed K3)	
In plain words	Each Foundation Level test step has a **security version** with its own goal. You must be able to look at an activity and name **which step it belongs to**.
How it works	**Planning and control**: define a **scope that corresponds to the security risks**. Set objectives, resources, estimates, schedule, metrics, entry and exit criteria. **Monitor** progress and **act** on what other activities report. **Analysis and design**: understand the **specific threats and risks**, using **risk assessments, audits and known-vulnerability sources**. **Review the test basis** (risk assessments, security requirements, policies) and define **test conditions**. Decide **which tools apply**. **Implementation and execution**: **turn conceptual tests into executable ones** (manual or automated), **run them from several perspectives** (**internal user, external user, malicious user**), and **set up the environment**. **Evaluating and reporting**: done **alongside execution**, so that **new threats are reported as soon as possible**. Determine the specific vulnerabilities, evaluate risk levels, and report interim and final results to **authorized** parties. **Closure**: make sure all planned tests ran and reports were delivered. **Archive testware, data and results securely**, so tests can be repeated. **Feed lessons back** into secure development.
Example	You already have conceptual tests ("verify an outside vendor cannot reach payroll data"). You now write step-by-step manual tests from them and run them while logged in as a vendor account. Writing and running them is **implementation and execution**. The conceptual tests themselves were produced in **analysis and design**.
Exam angle	Official Q11: "use conceptual tests as a basis for manual tests and execute them from an external vendor's perspective" → **implementation**. Keywords: **conceptual → executable**, **perspectives** → implementation and execution. **Conditions, objectives, test basis** → analysis and design. **Scope corresponding to risk, criteria, monitoring** → planning and control.
Don't confuse	The steps are **not necessarily sequential**: they follow the project's lifecycle, and evaluation runs **during** execution. The example tasks in the syllabus's table are **examples, not mandatory requirements**.

Security testing in different lifecycle models	
In plain words	The **same process** is fitted to whatever lifecycle the project uses. The risks for the tester differ by model.

Security testing in different lifecycle models	
How it works	**Sequential (e.g. V-model):** security needs are written **early** in the requirements. The danger is that they **change but the documents do not**, so tests look complete but are **out of date**. Testing often happens **late**, when fixes are expensive. **Iterative / Agile:** needs **emerge in every iteration** (user stories, acceptance criteria, prototypes) and should be **tested in the iteration where they appear**. The danger is that some risks **cannot be fully tested in one short cycle**. **COTS (bought software):** usually **black-box**. **No source code → no structural analysis or structural testing.** Expect frequent **patches**. **Open source:** a variant of COTS, but **anyone can read the code**, including attackers. Once a vulnerability is **public**, everyone running that version or older is at risk, so **patch promptly**.
Example	In the syllabus's iterative example, the first four-week iteration delivers **four security functions used by everything after it**: login, SSL enablement, lost-password reset, and lockout after three failures. The tester works with the developer and tests each one **as it is built**. Each later iteration confirms the controls still work and that no new vulnerabilities appeared.
Exam angle	If a stem has no source code (COTS), any answer involving **code review or structural coverage** is wrong. If the stem leaves security testing until **acceptance**, that is the trap the syllabus warns about: it is **especially costly and risky** at the end.
Don't confuse	**"Can be done any time but commonly late"** describes sequential projects. It is a risk to avoid, not a recommendation.

2.3 Planning

What a security test plan must contain: the six elements (AS-3.2.1, guaranteed K4)	
In plain words	Security testing has **two aims**: confirm the **designed defenses are implemented and work**, and confirm that **no new vulnerabilities were introduced** during development. Both start from a **risk analysis**. A plan that meets the syllabus minimum answers **six questions**.
How it works	**1 Scope**: what is in and out? What is **achievable** with these resources, risks and time? **2 Who**: do we have the **skills**? Will we **outsource**? For vendor software, **which tests are the vendor's and which are ours**? Is **tool training** needed? **3 Schedule**: what must be tested **before** other testing can start (**logins and access rights**)? When will features be ready? How long will it take? **4 Tasks and time for each**: time to **design** tests, to **evaluate and report**, to run **security regression**, and to **build the environment**. **5 Environment**: its extent (platform, technology, size, location), whether it is **new**, and which **tools** must be installed. **6 Authorizations and approvals**: **who** approves, **when**, and is the **budget** enough?
Example	A plan lists scope, roles, vendor/internal responsibilities, a **high-level** schedule, environment set-up, and approvals. Ticking the six elements shows that **element 4 is missing**: nothing says **how long** it will take to design, run and evaluate the security tests. A high-level schedule (element 3) is not the same thing.
Exam angle	Official Q12 (3 points) is exactly this example. **Method: tick the six elements against the stem; the unticked one is the answer.** Reject things that are useful but **not in the minimum** (tester credentials, a copy of the regulations, breach contact lists).
Don't confuse	**Schedule** (element 3: order and calendar) vs **tasks and time** (element 4: the effort for each kind of task). Exam options use this difference.

Reviewing a security test plan	
In plain words	A plan is reviewed for **completeness and correctness**, like any other test document. Because security plans are technical, the review is usually **technical**, and it needs people who know the **policies**.
How it works	Review types: a **technical review** is most appropriate. **Walkthroughs and inspections** can also work. Use a **standard checklist** (for example, the six elements). Feedback must be **constructive** and **not aimed at the author**. Reviewers come from **all impacted areas**, not only security testers. A **business-unit manager** may know risks the testers do not. **IT auditors and security administrators** are especially useful because they know the **policies and procedures**.
Example	The finance manager reviewing a plan notices it never mentions the month-end batch that moves all payments. The testers did not know about it. That one comment adds the highest-risk process to the scope.
Exam angle	If asked who should review, prefer the option that **includes IT auditors and security administrators** and business people, over "only the security test team".
Don't confuse	A plan review is **not** the plan-element check itself. The six elements are **what** must be present. The review is **how** you find out whether they are present and correct.

2.4 Design

From risks to concrete tests: the basis and attributes of security test design (AS-3.3.1)	
In plain words	Design turns **risks, threats and requirements** into **conceptual tests**. Design can start from **any** of three bases, and the resulting tests must have five qualities.
How it works	**Starting points (any of these is workable):** a performed **risk analysis**, an available **threat model**, or an **ad hoc categorization** of where the security risks originate. **Functional vs structural security risks: functional** risks abuse what the system does (**SQL injection, account harvesting, password cracking**). **Structural** risks abuse how it is built (**buffer overflow**, getting in through a memory failure). **Five attributes of detailed security tests: prioritized** (by risks and threat models) · **traced** (to security requirements) · **audience-defined** (developers, functional testers, security testers) · **based on security defect profiles** · **designed to be automated where applicable**. **Design flow:** test approach → risks, threat models and requirements → design techniques → test cases and scenarios. **A design process must be able to produce tests from any identified risk, requirement or threat.**
Example	Threat model for an e-commerce login: account harvesting is ranked high. Conceptual test: "With a valid user name and a wrong password, the error message must **not reveal which of the two was wrong**." It is **prioritized** (high risk), **traced** (to requirement SEC-LOGIN-04), written **for the functional testers**, based on the **harvesting defect profile**, and **automatable**.
Exam angle	K3 design questions give an approach and some risks and ask for the right conceptual test. Choose the test that **targets the stated risk directly**. Classify the risk first: a functional abuse gets a functional test, and a memory or code-structure flaw gets a structural test.
Don't confuse	**Account harvesting and password cracking are functional security risks** in the syllabus, even though they sound technical. **Buffer overflow is structural.**

The design categories: functional controls, access controls and the rest	
In plain words	The syllabus sorts what to test into categories. The three that are most often confused are **functional security controls**, **functional access controls** and **structural access controls**.

The design categories: functional controls, access controls and the rest	
How it works	**Functional security controls** (e.g. transaction controls): business rules that stop unauthorized **actions**. Example: a bank teller cannot approve a withdrawal over a limit **without the head teller's approval in the system**. **Functional access controls** (logins, passwords, tokens): is the **front door** applied correctly? Password policy enforced, the level of control right for the risk, resistant to cracking tools, and **error messages that do not help harvesting**. **Structural access controls** (user rights, encryption levels, authentication settings): how **rights** are configured behind the scenes by admins and DBAs. Test with **one test account per access level** (minimal, manager, admin) and verify that a minimal user **cannot** do manager tasks. **Other categories: secure coding practices** (mostly static analysis; dynamic tests for validation and error handling) · **operating system access** (can rootkits be planted?) · **language vulnerabilities** (no language is secure by itself; implementation matters) · **platform vulnerabilities** (patches applied promptly everywhere) · **external threats** (DoS, intrusion) · **internal threats** (often the greatest).
Example	Payroll system. Functional security control: a pay-rate change needs a second approver. Functional access control: lockout after three failed logins. Structural access control: a clerk account cannot open the salary-history table even through a report tool.
Exam angle	When an option describes **a business rule** → functional security control. **The login itself** → functional access control. **Rights configured by administrators** → structural access control.
Don't confuse	**WhiteHat 2014** found **negligible differences** in vulnerability rates between languages (.NET, Java, PHP…). An option claiming one language is inherently secure is wrong.

Denial of service and intrusion: the external threats	
In plain words	**DoS** makes a system **unavailable to legitimate users**. **Intrusion** means someone **breaks in**. The syllabus describes several forms of each, and a four-level way to test for DDoS.
How it works	**DoS**: overloading bandwidth, connections or services. **DDoS**: launched **indirectly** from many machines, through **amplification** or **botnets** (machines compromised by viruses or Trojans). **Amplification / reflection**: send a small request to a protocol that gives a **large answer** (DNS, NTP), with the **victim's address forged** as the sender, so the large answers hit the victim. **Resource exhaustion**: abuse an expensive function, for example by asking for **SSL key renegotiation** over and over until the server does nothing else. **Logical DoS**: abuse **intended** functionality. With **predictable user names and permanent lockout after three failures**, an attacker can lock out everyone and swamp the helpdesk. **Testing DDoS in four levels:** (1) no **known malware** on the machines · (2) the **IDS spots many requests from one machine** quickly · (3) **configurations** that allow abuse (SSL, web server, DNS) · (4) **logic defects** that allow DoS. **Intrusion methods:** social engineering, injection, account compromise (harvesting, password reset), known vulnerabilities, malware, insecure configuration, authorization defects, and **application-logic attacks** (doing shopping-cart steps **out of order** to get a discount).
Example	A ticket site permanently locks accounts after three wrong passwords, and user names are e-mail addresses that can be guessed from a public list. On sale day an attacker locks 50,000 accounts. Nothing was hacked, but the service is denied. That is a **logical DoS**, and level 4 of DDoS testing (logic defects) would have caught it.
Exam angle	Match the mechanism to the name: "forged source address + DNS" → **amplification**. "Repeated SSL renegotiation" → **resource exhaustion**. "Lockout of guessable accounts" → **logical DoS**.
Don't confuse	**Intercepting a transmission sent from inside the organization to another organization is not an intrusion. The syllabus calls it an internal breach.**

Test case, test procedure or exploratory test?	
In plain words	Security tests use the **same formats** as other tests (IEEE 829, ISO/IEC/IEEE 29119-3, tailored). Only the **target** and the **environment** differ. Choose the format by whether the **order of steps matters**.
How it works	**Test case**: the most standalone form. **No required order.** Usually **one condition** (e.g. one password-format rule). **Test procedure**: a **sequence** of steps. Use it **when order matters** (e.g. "recover lost password"). A **test script** is an **automated** test procedure. **Experience-based (exploratory) test**: conditions and expected results are **not defined in advance**, but the **conditions tested and actual results must be recorded**.
Example	"Password shorter than 12 characters is rejected" → a test case. "Lock the account, call the helpdesk, use the temporary password, set a new password, log in again" → a test procedure, because each step depends on the one before.
Exam angle	If the stem describes **state that carries from step to step** (lockout, reset, multi-step approval), the answer is a **test procedure**.
Don't confuse	Exploratory does **not** mean undocumented. What you tried and what happened must still be **recorded**.

Turning a policy or procedure into tests (AS-3.3.2, K3 pool)	
In plain words	When the **test basis is a policy or procedure**, a security test becomes almost a form of **security audit**: it checks that the documented steps really happen and really protect. The syllabus gives three goals and one worked procedure.
How it works	**Goal 1:** understand the **purpose and scope** of the policy or procedure. **Goal 2:** assess its **testability**. **Goal 3:** create tests that relate **directly** to it. **The worked lockout procedure:** (1) three failed logins → lockout message on the **third** · (2) contact IT and **verify identity** → temporary password to a **known email address** · (3) log in with it · (4) set a **new password that follows the policy** (a chance to test the password policy too) · (5) log out · (6) log in with the new password. **Not every policy is that testable.** "Audit records contain all audited events with a timestamp, traceable to an individual" → you must **trigger sample events** and **check what is logged**.
Example	A procedure says "after **three** failed attempts the account is locked". A test case that says "after **several** failed attempts" cannot verify the procedure, because "several" could be two or ten. **Exact numbers in the procedure are test data.**
Exam angle	Official Q13: reject any lockout test that says **"several"** attempts, and any test using passwords **kept in a notepad on the PC**. The correct one follows the procedure **exactly**: three attempts → message → temporary password → change → log out → log in.
Don't confuse	Policies are **a** test basis, but **should not be the only one**. Attacker-perspective tests are still needed.

2.5 Execution, evaluation and maintenance

The security test environment: isolated, complete, restorable (AS-3.4.1)	
In plain words	Security tests can **spread malware, corrupt servers and take networks down**. So the environment where they run must be **separated from everything else**, must **look exactly like production**, and must be **easy to reset**.

The security test environment: isolated, complete, restorable (AS-3.4.1)	
How it works	**Isolated** from other systems, **to the degree the malware risk requires**. Untrusted code (third-party, open source) needs an **isolated or virtual** environment. Low-risk functional security tests may run in a normal integrated environment. **Complete**: it mirrors the **production** environment in OS (**exact version and configuration**), network, middleware, desktops, mobile devices, databases, **access rights**, **browsers and plug-ins**, **co-existing applications**, and **data** (engineered, or **obfuscated** production data). **Restorable**: so tests can be **repeated** and the environment **recovered after corruption**.
Example	A tester wants to check whether a new PDF plug-in can be exploited. They use a **virtual machine** (isolated), with the **same OS build, browser version and user rights** as staff laptops (complete), from a **snapshot** they can roll back after each attempt (restorable).
Exam angle	Official Q14: the main characteristic → **mimics production in terms of access rights**. Reject: **closely tied to production** (it should **not** be connected), **old OS versions**, and **extra plug-ins not in production** (these cause **false positives and false negatives**).
Don't confuse	"**Mimics** production" is right. "**Connected to** production" is wrong. The environment should look the same but be separate.

Planning and approvals before ANY security test (AS-3.4.2, guaranteed K2)	
In plain words	Without **written, specific, prior permission**, a security test can be a **crime**. Approval protects the organization and the tester.
How it works	**Legality:** in almost all countries it is illegal to (try to) access systems without permission, and **in some countries even to possess certain security testing tools**. You need a **waiver from the system or data owner** and **approval from your management**. **Detection:** tests may set off the **IDS**, and the tester may look like a **malicious insider**. **Damage:** tests can cause **major failures and blackouts**, so the risks must be known and precautions taken. **Personal consequences:** without prior and **specific** authorization the tester can face **dismissal or prosecution**. **The authorization form:** the authorizing entity · the testers · the **statement of work** · the **dates** · details such as **source IP addresses and accounts** · **attestations** (the customer owns the system, has authority, **has backed up and tested the restore**, and understands the risks) · a **hold-harmless clause** for the testers · **signatures** of someone authorized to sign.
Example	A tester runs a port scan against a partner's server "just to check", without the partner's signed waiver. Even if nothing breaks, this may be an offence, and the partner's IDS reports an attack.
Exam angle	Official Q15: the significant concern about tools → **some countries prohibit the use of certain security testing tools**. A **fast-track** approval in a crisis is fine. A **complete bypass** of approval is not.
Don't confuse	Permission is obtained **before** testing, not after (this also decides a Chapter 6 question). And it must be **specific**: a general "go ahead" is not enough.

Evaluating a vulnerability and reporting it safely (AS-3.5.1, guaranteed K4)	
In plain words	When a test finds a vulnerability, you must work out **what it is, how far it reaches, how bad it could be, and how to fix it**, and you must report it **without creating a new risk**. A vulnerability report is a map for an attacker.

Evaluating a vulnerability and reporting it safely (AS-3.5.1, guaranteed K4)	
How it works	**Minimum incident report:** tester name · test environment · steps performed (**to reproduce it**) · **nature** · **extent** · **potential impact** · **suggested remediation**. **Same defect tool, special category:** use the normal incident system, but give security defects a **special category** and **restrict access**, especially when an independent tester reports into a widely visible tool, when a fix will take time, or when **insiders may be a threat**. **The IT auditor** can decide whether access to results should be restricted. Even the **final report** may need to be confidential, depending on whether the vulnerabilities have been fixed. **First response to a finding:** (1) **is it in production?** (2) **record it in the secure tracker** (3) **notify the developer** (4) **keep testing for similar defects** (the extent).
Example	While testing an **update** to a live e-commerce site, you find XSS in the search box. You check the **current production version**: it is there too. You log it in the restricted security category, tell the developer the same day, and spend the afternoon testing the other input fields for XSS, because one usually means more.
Exam angle	Official Q16 (3 points) is this example. Reject any option that **adds the vulnerability to a general stakeholder or weekly status report** (it publicizes the flaw), and any option that **skips notifying the developer**.
Don't confuse	**Nature** = what kind of flaw. **Extent** = where else it exists. **Impact** = how much harm it could cause. Options sometimes put one in place of another.

Maintaining the security test process (AS-3.6.1)	
In plain words	Attack techniques change **every day**, so a security test process that cannot change quickly becomes useless. Maintenance is mostly about **adding new kinds of tests for new kinds of threats**.
How it works	Design the process **to be easy to change**. **Add new test types** as new threat types appear. **Watch for new tools**, and evaluate which ones add **power and flexibility**.
Example	A new attack against file-upload parsers is published. A maintained process adds "malformed upload" tests to the regression suite within a week. A rigid process waits for next year's plan.
Exam angle	In K2 questions on maintenance, the reason is always **evolving technology and threats**. Avoid options that freeze the process or the test suite.
Don't confuse	Process maintenance (changing **how** you test) is not the same as **regression testing** after a code change. Regression is Chapter 4.

2.6 Fuzzy points clarified

Places where the wording is easy to misread

"Evaluating and reporting" runs alongside execution, yet Q11 says evaluation comes "after execution". Both are true. Each **individual** test is evaluated right after it runs, so evaluation happens **during** the execution period, not in a separate phase at the end.

"Security test approach" in planning vs in Chapter 2. It is the same thing: the project-level implementation of the strategy (Figure 3.1). Design starts **from** the approach.

"Isolated" is not absolute. The syllabus says isolated **depending on the level of malware risk**. Isolation is a must for untrusted code, but not for every test.

"Secret" test results vs "confidential" reports. The syllabus does not use classification levels here. It says reports may need **restricted access**, and the **IT auditor** decides whether to restrict them.

Q12's "high-level schedule" is a trap. It satisfies element 3 (schedule), but element 4 (the **time for each task**: design, run, evaluate) is still missing.

2.7 Chapter 3 keyword glossary

Keyword	Working definition for the exam
account harvesting	Identifying valid user names; once a name is known, only the password remains. Test: error messages must not reveal which item was wrong.
password cracking	Recovering passwords, e.g. by tools; access controls must be **resistant to password cracking software**.
social engineering	Exploiting human behaviour as an attack vector — listed as an intrusion method (detail in Chapter 6).
test approach	Project-level implementation of the strategy for a specific project.
test plan	Document of scope, approach, resources, schedule — for security: six key elements.
test process	Planning and control, analysis and design, implementation and execution, evaluating and reporting, closure.

Section 3 — Tables and Summaries

3.1 Which process step? — activity lookup

Activity in the stem	Step
Define scope corresponding to risks; estimates; entry/exit criteria; monitor	**Planning and control**
Review risk assessments and policies; define test conditions; tool applicability	**Analysis and design**
Turn conceptual tests into manual/automated tests; execute from internal/external/malicious perspectives; set up environment	**Implementation and execution**
Determine vulnerabilities from outcomes; evaluate risk levels; report interim/final results	**Evaluating and reporting**
Confirm all tests run; archive results securely; analyze to improve development	**Closure**

3.2 Plan review checklist (tick against the stem)

Element	Present?
Scope — in/out, achievable	☐
Who — skills, outsourcing, vendor vs customer, tool training	☐
Schedule — what first, when available, how long	☐
Tasks and time — design, evaluate/report, regression, environment	☐
Environment — extent, new?, tools	☐
Authorizations — who, when, budget	☐

3.3 Test case or test procedure?

Test object	Form	Why
Password formatting rules	**Test cases**	Single, independent conditions
Recover lost password	**Test procedure**	**Sequence matters**
Three-try lockout and reset	**Test procedure**	Ordered steps with state
Probing an unknown admin page	**Exploratory**	Record conditions and actual results

3.4 DoS family

Type	Abuses	Example
Volumetric DoS	Bandwidth / resources	Nuisance requests
DDoS (botnet)	Many compromised machines	Trojans, viruses
Amplification	Protocol responses (DNS, NTP) + spoofed source	Full DNS record list sent to victim
Resource exhaustion	Expensive function	SSL renegotiation
Logical DoS	Intended function	Lockout of guessable accounts

3.5 First response to a found vulnerability

Order	Action
1	**Check whether it exists in production**

Order	Action
2	**Document in a secure defect tracking system** (special category, restricted access)
3	**Notify the developer** (and management as appropriate)
4	**Continue testing for similar defects** — determine **extent**
Never	Put it into a **general stakeholder or weekly status report**

Reproduce from memory before moving on

Table 3.1 of the syllabus (five steps × goal) — guaranteed K3.
The six plan elements — guaranteed K4.
The incident report minimum and table 3.5 — guaranteed K4.
Isolated, complete, restorable; **why approval** — K2s.

Section 4 — Demo Questions with Answers

*Questions 1–6 are the **official ISTQB CTAL-SEC Sample Exam questions for Chapter 3** — #11 to #16 — with official answers and rationale. LO tags are inferred and match the Exam Structure Table. Questions 7–26 are derived strictly from the syllabus. Correct options are marked ✔.*

Part A — The 6 official Chapter 3 questions

Q1. OFFICIAL · Sample Exam #11 (2 pts)

As a Security Administrator you are responsible for all aspects of the security process, including testing. For this particular process you are to use conceptual tests as a basis for manual tests and execute these from an external vendor's perspective. Which security testing process does this most parallel?

a) Security test creation of conditions and objectives

b) Security test implementation ✔

c) Overall evaluating and reporting of security testing

d) Security test analysis and design

Correct answer: b | LO: AS-3.1.1 (K3) | Syllabus §3.1.1

Why: (b) — official rationale: **the use of the conceptual tests to create the manual tests and perform the execution is part of the security test implementation.** (a) and (d) **have already been done with the creation of the conceptual tests**. (c) **will occur after the tests have been executed**. Syllabus Table 3.1: implementation and execution **translates conceptual tests into tests that can be executed either manually or with tools … using a variety of security test perspectives – internal user, external user, malicious user**.

Q2. OFFICIAL · Sample Exam #12 (3 pts)

You have been developing a security test plan for a system that will store medical information for patients and will transfer that data to specialist doctors. You have covered the following areas in your plan:

- Scope (what's in scope and out of scope)
- Roles and assignments
- Responsibilities (vendors vs. internal)
- High level schedule
- Environment requirements and setup
- List of necessary authorizations and approvals

What information do you still need to supply in this test plan to meet the minimum requirements as noted in the syllabus?

a) A list of the necessary credentials and training for the personnel who will be conducting the testing

b) A schedule showing the time that will be required to design, run and evaluate the security tests ✔

c) A copy of the regulatory standards that must be met by this system

d) A list of the individuals who will be doing the testing and their contact information in the event of a security breach

Correct answer: b | LO: AS-3.2.1 (K4) | Syllabus §3.2.2

Why: (b) — official rationale: **correct per the syllabus**. (a) **might be needed but that is not one of the minimum requirements and may already be understood in the roles and responsibilities section**. (c) **the standards might be referenced but not included in the plan**. (d) **this level of detail does not belong in the plan and the individual testers should not be contacted during a breach**. Syllabus §3.2.2 element: **defining the tasks to be performed and the time required for each** (design, evaluate and report, regression, environment).

Q3. OFFICIAL · Sample Exam #13 (2 pts)

Which of the following test cases would best test a system's security procedure?

a) Three unsuccessful login attempts will generate a lock-out message. Contact your manager or the System Administrator so they can give you a temporary password over the phone. You must then change the temporary password upon logging in. You log out then log back in using your newly created password. ✔

b) You receive a lock-out message after several attempts to log in. You call IT support to obtain a new password. You log in with the temporary password, log back out, then log in again and enter a new password.

c) After several attempts you are locked out of the system. You use a password that worked previously. However, it no longer works. You attempt to create a new password but you are now locked out. A complete reboot of the machine is the next step to take you to the prompt to re-enter the password.

d) After the first attempt to use an invalid password you immediately pull up a list of passwords on your notepad on your PC to ensure you are using the correct one. You try another password from the list and it works.

Correct answer: a | LO: AS-3.3.2 (K3) | Syllabus §3.3.2

Why: (a) — official rationale: **B and C are not correct because of the word "several". D is not correct because this would definitely not be a good security practice.** Syllabus §3.3.2 example: three failed attempts → lock-out message → identity verified and temporary password issued → log in with temporary password → create a policy-conformant new password → log out → log in with the new password.

Q4. OFFICIAL · Sample Exam #14 (1 pt)

Which of the following are main characteristics of an effective security test environment?

a) Closely tied to production systems to enhance security at all points

b) Isolates different old versions of the operating systems for use in the environment

c) Mimics the production environment in terms of access rights ✔

d) Includes all production environment plug-ins as well as other plug-ins not in the production environment in order to ensure the most comprehensive setup

Correct answer: c | LO: AS-3.4.1 (K2) | Syllabus §3.4.1

Why: (c) — official rationale: **the closer the test environment mimics production, the more valid the testing will be. This is particularly true when it comes to access rights and delegation settings.** (a) **the systems don't need to be and probably shouldn't be connected**. (b) **may be useful, but is not a main characteristic**. (d) **includes plug-ins that are not in production which could result in both false positives and false negatives**.

Q5. OFFICIAL · Sample Exam #15 (1 pt)

What is a significant concern when seeking approval for the security testing tools?

a) Some countries prohibit the use of certain security testing tools ✔

b) Ensure the approval process for security testing tools can be bypassed on an exception basis in cases where a malicious event is in progress

c) The risks of the tool are rarely known before it is procured and are better discovered when the tool is in use

d) Because security testing tool risks are usually known, there is no need for a mitigating strategy

Correct answer: a | LO: AS-3.4.2 (K2) | Syllabus §3.4.2

Why: (a) — official rationale: **while some tools are quite good and effective for testing, they may be prohibited by some countries and some organizations.** (b) **there is always a danger of deploying a sub-optimal tool to deal with a crisis. A fast-track approval process makes sense, but a complete bypass is risky.** (c) and (d) **there may be unknown risks from tools and it's better to do the due diligence in tool selection**. Syllabus §3.4.2: **in some countries it is even against the law to have access to security testing tools**.

Q6. OFFICIAL · Sample Exam #16 (3 pts)

You are reviewing a set of security test results run on a product that is going through final testing before release to production. This is an update of a version that is currently in production. The application just tested was your e-commerce site, and it has a defect that allows cross-site scripting. Which of the following is the proper set of steps you should take?

a) Report the problem to the developer, add it to the stakeholder report, and continue testing for other types of defects

b) Test if the problem exists in the current production version, document the defect in a secure system, notify the developer, continue testing for other XSS defects ✔

c) Investigate the extent of the problem by conducting further tests on the planned release with particular concentration on other XSS issues, conduct static analysis on the code

d) Inform management, document the defect and include it in your weekly status report to stakeholders, continue testing for other security defects to determine the extent of the security issues

Correct answer: b | LO: AS-3.5.1 (K4) | Syllabus §3.5

Why: (b) — official rationale: **the first priority is to see if the problem exists in the production version. The defect should be documented only in a secure defect tracking system since the problem may exist in production. Since one XSS issue was found, there may be others so continued testing is warranted.** (a) **the defect should not be publicized in the stakeholder report**. (c) **while further testing is needed, notification is critical**. (d) **because of the stakeholder reporting**.

Part B — 20 additional syllabus-derived questions

*The K-level in each question's label is the level of the **learning objective** it practises, not of the question itself. Some Part B items are quick recall checks on a K3 or K4 objective. Use them to learn the facts, and use the scenario items (and Part A) to practise the reasoning the real K3/K4 questions need.*

Q7. Derived from syllabus

A tester reviews the latest security risk assessment, the security requirements and the password policy, and then defines test conditions for each identified risk and the tools applicable to each. Which security test process step is this?

a) Security test planning and control

b) Security test implementation and execution

c) Evaluating and reporting results of security testing

d) Security test analysis and design ✔

Correct answer: d | LO: AS-3.1.1 (K3) | Syllabus §3.1.1

Why: (d). Table 3.1: analysis and design tasks are **review items that serve as the basis (risk assessments, requirements, policies)** and **define security test conditions** based on objectives, risks, standards and known vulnerabilities, defenses, scope and **applicability of security test tools**.

Q8. Derived from syllabus

At the end of a release's security testing, the team confirms that all planned security tests were performed, that reports were delivered, and archives the results and test data in a secure location. Which additional task belongs to the same process step?

a) Analyze security test results to improve system and application development in terms of security ✔

b) Define security test estimates and schedules

c) Set up the security test environment

d) Define entry and exit criteria

Correct answer: a | LO: AS-3.1.1 (K3) | Syllabus §3.1.1

Why: (a) is a **test closure** task in Table 3.1, alongside ensuring all tests were run, confirming deliverables and **archiving results, test data and other sensitive information in secure locations**. (b) and (d) are planning; (c) is implementation.

Q9. Derived from syllabus

A company plans to deploy a commercial off-the-shelf HR package without customization. Which security testing activity is NOT possible?

a) Black-box testing of access rights

b) Verifying that vendor security patches are applied

c) Structural analysis and structural testing of the code ✔

d) Testing the configuration of the package in the target environment

Correct answer: c | LO: AS-3.1.1 (K3) | Syllabus §3.1.2

Why: (c). §3.1.2: COTS projects **are often black-box in nature ... There is no access to the code, so structural analysis and structural testing is not possible.** They often require **frequent security updates and patches**.

Q10. Derived from syllabus

In an Agile project, a security risk is identified during sprint 4. What does the syllabus say about addressing it?

a) It must be deferred to a dedicated security sprint at the end of the project

b) It can (should) be addressed in the iteration in which it is identified, although some risks may not be fully mitigated and tested within one short release cycle ✔

c) It must be documented in the software requirement specification before any testing

d) It should be handed to the acceptance testers

Correct answer: b | LO: AS-3.1.1 (K3) | Syllabus §3.1.2

Why: (b) — §3.1.2 iterative/incremental: **security needs and risks may change during the project, and can (should) be addressed in the iteration in which they are identified ... it may not be possible to completely mitigate and test it during one short release cycle**.

Q11. Derived from syllabus

A security test plan for a vendor-supplied payroll module contains: scope and exclusions; a named in-house tester with tool training; a schedule that tests logins and access rights first; time estimates for design, execution, reporting, regression and environment build; and a description of an isolated test environment with the scanning tools to be installed. Reviewers note that the vendor has a contract clause stating it performs its own penetration tests.

Which feedback identifies the most significant weakness of the plan?

a) The schedule should test logins last, after functional testing

b) Tool training should be removed, as testers are expected to know the tools

c) The environment should be shared with production to save cost

d) The plan does not state who must authorize and approve the tests, when authorization is needed, or whether budget is sufficient — and it does not clarify which tests are the vendor's responsibility and which are the customer's ✔

Correct answer: d | LO: AS-3.2.1 (K4) | Syllabus §3.2

Why: (d). Checking the six elements: scope ✓, who ✓ (but **vendor vs customer responsibility** is unresolved despite the contract clause), schedule ✓ (**access rights and logins first** is a strength), tasks and time ✓, environment ✓, **authorizations and approvals ✗**. (a) and (c) would turn strengths into weaknesses; (b) contradicts the syllabus question **do security testers need training in specific tools?**

Q12. Derived from syllabus

Who should be included in the review of a security test plan, according to the syllabus?

a) Knowledgeable people from all areas impacted by the security aspects of the plan — not necessarily security experts — with IT auditors and security administrators being especially helpful ✔

b) Only certified security testers, to keep the plan confidential

c) Only the author of the plan and the test manager

d) External penetration testers who will execute the tests

Correct answer: a | LO: AS-3.2.1 (K4) | Syllabus §3.2.2

Why: (a) — §3.2.2: the review team should include **knowledgeable people from all areas impacted**; members **may not necessarily be security testers**; e.g. **a business-unit manager** may know risks; **IT auditors and security administrators are especially helpful** because of their knowledge of policies and procedures.

Q13. Derived from syllabus

What are the two aspects security testing should generally focus on, according to §3.2.1?

a) Performance under attack and recovery time

b) Compliance with ISO 27001 and PCI

c) Verifying designed defenses are implemented and function as designed, and verifying that no vulnerabilities are introduced during development ✔

d) Finding all vulnerabilities and proving the system is secure

Correct answer: c | LO: AS-3.2.1 (K4) | Syllabus §3.2.1

Why: (c) — verbatim. Testing for introduced vulnerabilities **starts with an assessment of the practices used by the development team**.

Q14. Derived from syllabus

Which test best addresses the risk of account harvesting on a login page?

a) Enter a valid user name and valid password and confirm access

b) Enter a correct user name with an incorrect password and verify the error message does not indicate which of the two items is incorrect ✔

c) Enter 1,000 random user names to measure response time

d) Verify that passwords are stored hashed

Correct answer: b | LO: AS-3.3.1 (K3) | Syllabus §3.3.1

Why: (b) is the syllabus's example: **account harvesting is the practice of identifying a user name … A common test is to verify that when a correct user name is entered with an incorrect password, the error message does not indicate which of the items is incorrect.**

Q15. Derived from syllabus

An application has three roles: clerk, supervisor and administrator. Which conceptual test targets structural access controls?

a) Verify the login page uses TLS

b) Verify that password rules are enforced

c) Verify that a withdrawal above a limit requires a second approval

d) Create a test account for each access level and verify that a clerk cannot perform supervisor-level or administrator-level actions ✔

Correct answer: d | LO: AS-3.3.1 (K3) | Syllabus §3.3.1

Why: (d) — §3.3.1: tests of structural access controls include **creation of test user accounts for each level of security access and verifying that each level of access does not have access rights that are restricted for that level**. (c) is a **functional security control** (transaction control); (b) a **functional access control**.

Q16. Derived from syllabus

Which of the following is a secure coding practice listed in the syllabus?

a) Run services with the least possible privileges, never under root, each with its own operating system account ✔

b) Display the stack trace to users so they can report errors accurately

c) Allow the application to issue commands directly to the operating system for efficiency

d) Pass sensitive information in HTTP GET requests so it can be bookmarked

Correct answer: a | LO: AS-3.3.1 (K3) | Syllabus §3.3.1

Why: (a) is listed. The syllabus says the opposite of (b) (**error handlers should not display stack trace**), (c) (**should not issue commands directly to the OS; use built-in APIs**) and (d) (**HTTP GET requests should not include sensitive information**).

Q17. Derived from syllabus

An attacker discovers that user names follow the pattern firstname.lastname and that accounts are permanently locked after three failed logins. The attacker locks hundreds of accounts. What type of attack is this?

a) Amplification attack

b) Resource exhaustion attack

c) Logical denial of service attack ✔

d) Man-in-the-middle attack

Correct answer: c | LO: AS-3.3.1 (K3) | Syllabus §3.3.1

Why: (c) — §3.3.1's own example of a **logical DoS**: an attacker **abuses intended functionality** (predictable user names + permanent lockout) to prevent users accessing the system, **indirectly DoS-ing the helpdesk**.

Q18. Derived from syllabus

Which of the following is one of the four levels of testing for DDoS described in the syllabus?

a) Measure maximum throughput of the web server under expected load

b) Test the ability of the intrusion detection systems to quickly identify multiple requests from a single computer in a short period of time ✔

c) Verify that all users can reset their passwords

d) Run a code review of the login page

Correct answer: b | LO: AS-3.3.1 (K3) | Syllabus §3.3.1

Why: (b). The four levels: **(1) computers not infected with known malware; (2) IDS quickly identifies multiple requests from one computer; (3) identify configurations allowing abusable functionality (SSL, web server, DNS); (4) identify logic defects that can allow a DoS.**

Q19. Derived from syllabus

Which situation does the syllabus explicitly state is NOT an intrusion attack?

a) Intercepting a network transmission sent from inside an organization to someone in another organization ✔

b) Performing steps out of order in a shopping cart to obtain a discount

c) Injecting SQL into a login field

d) Exploiting a known firewall vulnerability

Correct answer: a | LO: AS-3.3.1 (K3) | Syllabus §3.3.1

Why: (a) — §3.3.1: **intercepting a network transmission sent from inside an organization to someone in another organization is not considered an intrusion attack, but rather an internal breach.** (b) is an **application logic attack**, listed as an intrusion method.

Q20. Derived from syllabus

For testing the "recover lost password" process, which form of test description is most appropriate?

a) Independent test cases with no defined order

b) An exploratory charter with no recorded results

c) A single test case covering one condition

d) A test procedure, because the sequence of steps is important ✔

Correct answer: d | LO: AS-3.3.1 (K3) | Syllabus §3.3.1

Why: (d) — §3.3.1: **test procedures are used when sequence is important. For example, a test procedure would be helpful in testing the "recover lost password" process.** Even exploratory tests should **record the conditions tested and actual results**.

Q21. Derived from syllabus

A policy states: "Audit records contain all audited events with date/time stamp and are traceable to specific individuals." Which test approach does the syllabus describe for this kind of policy?

a) Declare the policy untestable and exclude it

b) Review the vendor's documentation only

c) Trigger a sample set of events to be logged and verify the logged information — such as user ID and date/time stamp — is correct ✔

d) Delete the audit log and check whether it regenerates

Correct answer: c | LO: AS-3.3.2 (K3) | Syllabus §3.3.2

Why: (c). §3.3.2: **while not impossible to test**, actions **would need to be performed to trigger a sample set of events to be logged ... and the accuracy of the information logged would need to be verified as correct, such as the user ID and the date and time stamp**.

Q22. Derived from syllabus

What are the goals of designing tests to validate security policies and procedures?

a) Understand the purpose and scope of the policy or procedure, assess its testability, and create tests related directly to it ✔

b) Rewrite the policy so that it is testable, then hand it to developers

c) Replace other perspectives of security testing with policy-based tests

d) Prove that the policy is legally compliant

Correct answer: a | LO: AS-3.3.2 (K3) | Syllabus §3.3.2

Why: (a) — §3.3.2. The syllabus also warns that **security policies and procedures should not be the only basis of testing**, contradicting (c).

Q23. Derived from syllabus

A team must security-test an untrusted open source component of unknown provenance. Which environment is advisable?

a) The shared integration environment, because it is cheapest

b) An isolated or virtual test environment, because of the risk of malware corrupting a server or network ✔

c) The production environment, for realism

d) A developer's laptop

Correct answer: b | LO: AS-3.4.1 (K2) | Syllabus §3.4.1

Why: (b) — §3.4.1: **when testing unknown and untrusted code, the possibility of malware corrupting a server and/or network makes it advisable to test in an isolated or virtual test environment**.

Q24. Derived from syllabus

Which item should an authorization form for security testing contain?

a) The tester's home address and date of birth

b) The list of vulnerabilities the tester expects to find

c) An attestation that the customer has backed up all systems and data and has tested that the system can be restored ✔

d) A guarantee that no outage will occur

Correct answer: c | LO: AS-3.4.2 (K2) | Syllabus §3.4.2

Why: (c). Attestations: customer **owns** the system, has **authority** to authorize, has **performed a backup**, has **tested restore**, **understands the risks** — plus a **hold harmless** clause and **signatures**. Tests **may** lead to outages, so (d) cannot be guaranteed.

Q25. Derived from syllabus

An independent security testing firm finds a privilege-escalation flaw in a bank's internal portal. The bank's defect tracker is visible to all 400 IT staff and contractors. The fix is scheduled for next quarter.

What is the best way to record and report this finding?

a) Log it in the defect tracker with normal visibility so all staff can avoid the affected screens

b) Report it only verbally to avoid a paper trail

c) Publish it in the weekly project newsletter so stakeholders are informed

d) File an incident report with nature, extent, impact and suggested remediation under a special security category with restricted access — the IT auditor can decide on access ✔

Correct answer: d | LO: AS-3.5.1 (K4) | Syllabus §3.5

Why: (d). All three syllabus triggers for securing results apply: **independent organization** reporting into a tool **with few restrictions**, a vulnerability **not immediately resolved**, and **internal personnel may be a potential threat**. Reports should have **a special category** and may need to be **secured**; **the IT auditor should be able to make the determination** on restricting access.

Q26. Derived from syllabus

Why must the security testing process be designed to change easily?

a) Because security testing targets and threats change daily, and maintaining the process often means adding new types of tests for new types of threats ✔

b) Because ISTQB updates the syllabus every year

c) Because the test team rotates every release

d) Because regulators require a new process for every project

Correct answer: a | LO: AS-3.6.1 (K2) | Syllabus §3.6

Why: (a) — §3.6: **the security testing targets and threats change daily, so the security testing process needs to be designed to change easily**; testers should also **keep up with new tools**.

Most Important Questions From This Chapter

*Ranking rationale: guaranteed questions on **AS-3.1.1 (K3), 3.2.1 (K4), 3.4.2 (K2), 3.5.1 (K4)**; one K3 from 3.3.1/3.3.2; one K2 from 3.4.1/3.6.1.*

Rank	Topic / question focus	LO	Why it matters
1	Evaluate, secure and report vulnerabilities — Q6, Q25	AS-3.5.1	**Guaranteed 3 points**; production first, secure tracker, never the stakeholder report.
2	Plan review against six elements — Q2, Q11, Q12, Q13	AS-3.2.1	**Guaranteed 3 points**; tick-list method.
3	Process step recognition — Q1, Q7, Q8, Q9, Q10	AS-3.1.1	**Guaranteed 2 points**; Table 3.1.
4	Policy-based and conceptual test design — Q3, Q14–Q22	AS-3.3.1 / 3.3.2	**2 points**; exact numbers, account harvesting, access levels, procedures.

Rank	Topic / question focus	LO	Why it matters
5	Approvals — Q5, Q24	AS-3.4.2	**Guaranteed 1 point**; legality, IDS, outages, form contents.
6	Environment and maintenance — Q4, Q23, Q26	AS-3.4.1 / 3.6.1	**1 point**; isolated, complete, restorable.

Section 5 — Exam Tricks

5.1 Frequently tested topics (evidence-based)

LO	K	Qs	Pts	Sample #	Format used officially
AS-3.1.1	**K3**	1	2	#11	Activity described → **which process step**?
AS-3.2.1	**K4**	1	**3**	#12	Plan contents listed → **what is still missing**?
AS-3.3.x	**K3**	1	2	#13	Four test cases → **which best tests the procedure**?
AS-3.4.1 / 3.6.1	K2	1	1	#14	Main characteristic of an effective environment
AS-3.4.2	K2	1	1	#15	Concern when seeking approval for tools
AS-3.5.1	**K4**	1	**3**	#16	XSS found in an update → **proper set of steps**
Chapter 3 total	—	**6**	**12**	**#11–#16**	**15% of the points; two 3-point K4s**

The arithmetic

Chapter 3's two K4s and one guaranteed K3 total **8 points** — two-thirds of the chapter, and as much as **eight K2 questions** anywhere else on the paper. The plan-review and incident-handling questions are **pattern questions**: once you have the six-element checklist and the "production first, secure tracker" sequence, they are fast.

5.2 Common distractors observed

- **Publicising a vulnerability** — stakeholder report, weekly status report. Always wrong while unresolved.
- **Investigating without notifying** — further tests and static analysis, but no notification. **Notification is critical.**
- **Plausible plan extras** — credentials and training lists, copies of regulations, tester phone numbers. Not among the six minimum elements.
- **"Several" attempts** in a lockout test case — the procedure says **three**.
- **Environment tied to production** or with **extra plug-ins** — false positives and negatives.
- **Approval bypass in a crisis** — a fast track is fine, a complete bypass is risky.
- **Choosing analysis/design** when conceptual tests **already exist** — the next step is implementation.

5.3 Keyword recognition (map words → answers)

If the stem says...	Think...
"conceptual tests → manual tests, execute"	**Implementation and execution**
"define scope corresponding to risks"	**Planning and control**
"review risk assessment, define conditions"	**Analysis and design**
"archive results securely"	**Closure**
"what is still missing from the plan"	Tick the **six elements**
"version currently in production"	**Check production first**
"found one XSS"	**Continue testing for others** (extent)
"untrusted code", "third-party"	**Isolated / virtual** environment
"mimics production"	**Complete** — incl. **access rights**
"against the law", "prohibited tools"	**Approval / authorization**
"hold harmless"	**Authorization form**

If the stem says...	Think...
"COTS"	**No structural testing** possible
"open source", "publicized vulnerability"	**Patch promptly**
"wrong error message reveals user name"	**Account harvesting**
"steps out of order for a discount"	**Application logic attack**

5.4 Typical traps

- **Functional vs structural access controls.** Functional = logins, passwords, tokens (does the login behave correctly?). Structural = **rights configured** per user level, encryption levels, authentication — test accounts per level.
- **Functional security controls** are **transaction controls** (head-teller approval) — a third, separate category.
- **Internal breach vs intrusion.** Intercepting outbound transmissions is an **internal breach**.
- **Test case vs procedure vs script.** Script = **automated procedure**.
- **Sequential lifecycle** — tests *can* be done any time but are **commonly late**; that is a risk, not a rule.
- **DDoS vs amplification vs logical DoS** — botnet vs protocol reflection vs abusing intended function.

5.5 How to eliminate wrong answers

7. In incident questions, **strike any option that widens the audience** of an unresolved vulnerability.
8. Then **strike any option that omits notification** or omits **checking production**.
9. In plan questions, **strike options the stem already covers**, then strike items that are **referenced, not included** (standards) or **too detailed** (phone numbers).
10. In design questions, **strike vague quantifiers** ("several", "a few") and **insecure practices** (passwords in notepad).
11. In process-step questions, ask **what already exists** — the answer is the **next** step.

5.6 Scenario-solving shortcuts

Scenario shape	Shortcut
Vulnerability found during final testing of an update	**Production? → secure tracker → notify → extent**
Plan with a bullet list	**Six-element tick list**
Policy text quoted	**Purpose/scope → testability → direct tests** with the policy's exact numbers
Role-based system	**Test account per level**, verify restrictions
E-commerce risks listed	SQLi, account harvesting, password cracking = **functional**; buffer overflow = **structural**
Third-party or open source code to test	**Isolated/virtual, complete, restorable**

5.7 Common candidate mistakes

- Learning Table 3.1 as five headings without the **goal sentence** for each — the goal sentences are what the questions paraphrase.
- Forgetting that **evaluation runs alongside execution** to report new threats **as soon as possible**.
- Assuming the plan review must be done by security experts only.
- Treating the secure coding list as optional reading — it is the source for Chapter 4's component tests too.

5.8 Mnemonics that carry the whole chapter

Mnemonic	Expands to
S-W-S-T-E-A (“Sweet Tea”)	Plan elements: **S**cope, **W**ho, **S**chedule, **T**asks and time, **E**nvironment, **A**uthorizations
I-C-R (“I See Red”)	Environment: **I**solated, **C**omplete, **R**estorable
P-T-D-A-A	Design attributes: **P**rioritized, **T**raced, audience-**D**efined, defect-profile **A**ligned, **A**utomatable
T-E-S + N-E-I-R	Incident report: **T**ester, **E**nvironment, **S**teps + **N**ature, **E**xtent, **I**mpact, **R**emediation
Prod → Secure → Tell → Search	Response to a found vulnerability
P-T-C	Policy-based test goals: **P**urpose & scope, **T**estability, **C**reate tests

5.9 Chapter 3 mind map

Branch	Leaves
Process	Five steps · goals · example tasks · not sequential · hierarchy · lifecycles (sequential, iterative, COTS, open source)
Planning	Two aims · six elements · technical review · IT auditors and security admins
Design	Risk/threat model/ad hoc · functional vs structural · five attributes · nine categories · secure coding list · DoS family · intrusions · internal threats · format
Policy tests	Purpose → testability → tests · lockout example · audit records example
Execution	Isolated · complete · restorable · approvals · authorization form
Evaluation	Incident report minimum · special category · IT auditor · confidential final report
Maintenance	Change easily · new tests for new threats · new tools

Section 6 — Revision Notes

6.1 One-page chapter summary

Process. Security testing is a lifecycle activity aligned with development: planning and control (scope matched to risk), analysis and design (understand threats from assessments, audits, vulnerability sources), implementation and execution (make conceptual tests executable; test from many perspectives), evaluating and reporting (alongside execution), closure (secure archive, improve development). Policy and strategy are organizational; approach and tasks are project-level. Sequential, iterative, COTS and open source each change what is possible.

Planning. Verify defenses work as designed and that no vulnerabilities are introduced. Plan: scope, who, schedule, tasks and time, environment, authorizations. Review technically with a checklist; include IT auditors and security administrators.

Design. Start from risk analysis, threat model or ad hoc categorization. Tests are prioritized, traced, audience-defined, defect-profiled, automatable. Cover functional and structural access controls, functional security controls, secure coding, OS, language and platform vulnerabilities, external (DoS family, intrusions) and internal threats. Policy-based tests: purpose and scope, testability, direct tests using the policy's exact values.

Execution and evaluation. Environment: isolated, complete, restorable. Never test without authorization. Incident reports carry nature, extent, impact and remediation; file them securely; check production first; notify; look for more. Maintain the process for constant change.

6.2 Key definitions to memorize

Term	Definition
Account harvesting	Identifying valid user names, leaving only the password to guess
Logical DoS	Abusing intended functionality to deny access to others
Amplification attack	Abusing protocol responses with a spoofed victim address to multiply traffic
Test procedure	Specifies the sequence of test case execution; a script is an automated procedure
Hold harmless clause	Authorization-form clause protecting the testing entity

6.3 High-priority concepts (study first)

12. Incident handling and confidentiality (**AS-3.5.1**, 3 pts).
13. Six plan elements (**AS-3.2.1**, 3 pts).
14. Process steps and goals (**AS-3.1.1**, 2 pts).
15. Policy-based and conceptual test design (**AS-3.3.x**, 2 pts).
16. Approvals (**AS-3.4.2**, 1 pt); environment and maintenance (**AS-3.4.1 / 3.6.1**, 1 pt).

6.4 Last-minute revision sheet

If you read nothing else

Plan → scope · Analysis/design → threats & conditions · Implementation → conceptual → executable · Evaluate alongside · Closure → secure archive.

Plan = Scope, Who, Schedule, Tasks+time, Environment, Authorizations.

Found a vulnerability in an update? **Production? → secure tracker → notify developer → test for more.** Never the stakeholder report.

Environment: isolated · complete (access rights!) · restorable. No extra plug-ins.

Approval: illegal otherwise · IDS · outages · prosecution. Some countries ban tools.

Lockout test: **exactly three**, temporary password, change it, log out, log back in.

Account harvesting: error message must not reveal which field is wrong.

COTS = no structural testing. Open source = patch once public.

6.5 Memorization checklist

✓	I can…
☐	state the goal and three tasks of each of the five process steps
☐	describe security testing concerns for sequential, iterative, COTS and open source
☐	list the six plan elements with one question each
☐	name the five design attributes and the four-step design flow
☐	give an example test for functional security controls, functional access controls and structural access controls
☐	list eight secure coding practices
☐	distinguish DoS, DDoS, amplification, resource exhaustion and logical DoS
☐	write the six-step lockout procedure
☐	list the three environment attributes and four approval reasons
☐	list the seven minimum incident report items and three reasons to restrict them

6.6 Exam-day quick review

- Two 3-point questions here: **plan gaps** and **vulnerability handling**.
- **What already exists?** → the answer is the next process step.
- **Exact numbers** beat “several”.